Strings for Regionals

Hashing tricks, Manacher, palindromes, and string DP

Learning goals

- Use polynomial hashing **safely** (mod $2^{61} - 1$, random base)
- Combine hashing with binary search: LCP, repeated substrings, comparisons
- Find all palindromic substrings in O(n) with **Manacher**
- Solve palindrome-structure problems (prefix palindromes, partitions)
- Run DP over a **KMP automaton** for "strings avoiding / containing a pattern"

Regional string problems

- Rarely "implement suffix automaton"
- Usually: a known tool + one observation
  - "compare substrings fast" → hashing
  - "palindromes" → Manacher or hashing both directions
  - "count strings of length n with/without p" → automaton DP
  - "split into pieces with property" → DP over prefixes
- Year Curriculum gave you KMP, Z, tries, Aho–Corasick: we build on them

Hashing refresher

```
H(s[0..i)) = s0·B^{i−1} + s1·B^{i−2} + … + s_{i−1}      (mod M)
H(s[l..r)) = h[r] − h[l] · B^{r−l}
```

- O(n) preprocessing, O(1) substring hash
- Equal strings ⇒ equal hashes; unequal ⇒ collision with prob. ≈ n/M per comparison
- The danger is **how many** comparisons you make

Choosing the modulus

Choice	Problem
mod 2^{64} (overflow)	Broken by Thue–Morse strings, *for any base*
one ~10^9 prime	Birthday paradox: 10^5 hashes in a set ⇒ collisions likely
two ~10^9 primes	OK, but 2× slower and more code
$2^{61} - 1$, random base	Safe and fast with `__int128`

- Random base chosen at runtime ⇒ no fixed anti-hash test

Mod $2^{61} - 1$ multiplication

```
const u64 MOD = (1ULL << 61) - 1;
u64 mulmod(u64 a, u64 b) {
    __uint128_t c = (__uint128_t)a * b;
    u64 r = (u64)(c & MOD) + (u64)(c >> 61);   // since 2^61 ≡ 1
    return r >= MOD ? r - MOD : r;
}
u64 get(int l, int r) {                          // hash of s[l..r)
    u64 v = h[r] + MOD - mulmod(h[l], pw[r - l]);
    return v >= MOD ? v - MOD : v;
}
```

Hashing trick 1: binary search on length

- Property monotone in length? → binary search + hash check
- Longest repeated substring: if length L repeats, so does L − 1

```
auto repeated = [&](int len) {
    unordered_set<u64> seen;
    for (int i = 0; i + len <= n; i++)
        if (!seen.insert(H.get(i, i + len)).second) return i;
    return -1;
};
```

- O(n log n); *AtCoder ABC 141 E Who Says a Pun?* (non-overlapping variant)

Hashing trick 2: LCP and comparison

- LCP of suffixes `i` and `j`: binary search the largest L with equal hashes → O(log n)
- Lexicographic compare of two substrings: LCP, then compare next character
- ⇒ sort suffixes / rotations with a comparator in $O(n \log^2 n)$
- Minimal rotation, comparing concatenations `a+b` vs `b+a`, etc.

Hashing trick 3: palindromes and updates

- Palindrome test: hash of `s[l..r)` = hash of the reversed substring (hash `rev(s)` too)
- *CF 7D Palindrome Degree*: prefix palindrome test, rolling both directions
- With point updates: store hashes in a **segment tree** (Lecture 6 mergeable nodes)
- *CSES 2420 Palindrome Queries*

Manacher: the idea

- Insert separators: `abba` → `#a#b#b#a#` (all palindromes become odd)
- `d[i]` = radius of the longest palindrome centred at i
- Keep the rightmost palindrome `[l, r]`; for i inside it, the mirror `l + r − i` gives a lower bound

```
   l          mirror       c        i          r
   [ . . . . ( . . . ) . . | . . ( . . . ) . ]
```

- Each successful extension moves r right ⇒ O(n) total

Manacher: code

```
string t = "#";
for (char c : s) { t += c; t += '#'; }
vector<int> d(t.size(), 0);
for (int i = 0, l = 0, r = -1; i < (int)t.size(); i++) {
    int k = (i > r) ? 0 : min(d[l + r - i], r - i);
    while (i - k - 1 >= 0 && i + k + 1 < (int)t.size()
           && t[i - k - 1] == t[i + k + 1]) k++;
    d[i] = k;
    if (i + k > r) { l = i - k; r = i + k; }
}
// palindrome in s: length d[i], starting at (i - d[i]) / 2
```

What Manacher gives you

- Longest palindromic substring (CSES 1111)
- Number of palindromic substrings: Σ ⌈d[i] / 2⌉ over the separated string
- "Is s[l..r] a palindrome?" in O(1): centre radius ≥ half-length
- Longest palindromic **prefix** / **suffix**: centres whose palindrome touches the border

Example: CF 1326D2 Prefix-Suffix Palindrome

> Find the longest palindrome t = a + b, a a prefix of s, b a suffix of s, |a| + |b| ≤ |s|

1. Strip the longest k with `s[i] = s[n−1−i]` for all i < k
2. Middle part m: answer = `s[0..k) + (longest palindromic prefix or suffix of m) + s[n−k..n)`
3. Longest palindromic prefix of m: prefix function of `m + '#' + reverse(m)`, or Manacher

- O(n); a typical "one observation + one tool" regional medium

String DP: splitting into pieces

- `dp[j]` = best for prefix of length j; transition over last piece `s[i..j)`
- Pieces = dictionary words → trie walk from i (CSES 1731 *Word Combinations*)
- Pieces = palindromes → `pal[i][j]` table or Manacher

```
cuts[j] = min over i ≤ j with s[i..j] palindrome of cuts[i−1] + 1
```

- $O(n^2)$ with the palindrome table; fine for n ≤ 5000

String DP: interval DP on palindromes

- *CF 245H Queries for Number of Palindromes*: count palindromic substrings of `s[l..r]`, many queries, n ≤ 5000
- `pal[i][j]` table, then `cnt[i][j] = cnt[i+1][j] + cnt[i][j−1] − cnt[i+1][j−1] + pal[i][j]`
- O(n²) precomputation, O(1) per query

DP over the KMP automaton

> Count strings of length n over Σ that **do not contain** p

- State = length of longest prefix of p that is a suffix of the text so far
- Transition `go[state][c]` from the prefix function
- Reaching state |p| = pattern found → drop (or absorb)

```
dp[i+1][go[s][c]] += dp[i][s]      for s < |p|, c ∈ Σ
contains = |Σ|^n − Σ_s dp[n][s]
```

- *CSES 1112 Required Substring*; many patterns → Aho–Corasick automaton

Building the automaton

```
for (int s = 0; s < m; s++)
    for (int c = 0; c < 26; c++) {
        if (p[s] - 'A' == c) go[s][c] = s + 1;
        else go[s][c] = (s == 0) ? 0 : go[pi[s-1]][c];
    }
```

- `pi[s−1] < s` ⇒ that row is already computed
- O(m·|Σ|) build, O(n·m·|Σ|) DP — matrix power if n is huge

Complexity

Technique	Time
Hash preprocessing / query	$O(n)$ / $O(1)$
Binary search + hashing	$O(n \log n)$
Manacher	$O(n)$
Palindrome table + partition DP	$O(n^2)$
KMP automaton DP	$O(n \cdot m \cdot$

Common pitfalls

- Hashing with mod 2^{64} or a fixed public base
- Character value 0 (`'a' - 'a'`) makes "a" and "aa" collide in some schemes → use the raw char code
- Forgetting `+ MOD` before subtracting hashes
- Manacher index mapping between `t` and `s`
- Automaton: not handling state m (pattern found)
- $O(n^2)$ palindrome tables with $n = 10^5$ → memory blow-up

Practice problems

- CSES 1753 String Matching · 1732 Finding Borders
- CSES 1111 Longest Palindrome · 2420 Palindrome Queries
- CSES 1112 Required Substring · 1731 Word Combinations
- AtCoder ABC 141 E — Who Says a Pun?
- Codeforces 1326D2 — Prefix-Suffix Palindrome (Hard version)
- Codeforces 7D — Palindrome Degree
- Codeforces 245H — Queries for Number of Palindromes
- Codeforces 271D — Good Substrings
- Codeforces 1200E — Compress Words

Summary

- Hash mod $2^{61} - 1$ with a random base; binary search on monotone properties
- Manacher gives every palindrome centre in O(n)
- Palindrome structure problems: strip, then longest palindromic prefix/suffix
- String DP: prefix DP over pieces; automaton DP for pattern avoidance
- Next: the mock regional